

NATIONAL UNIVERSITY OF SINGAPORE
DEPARTMENT OF ELECTRICAL & COMPUTER ENGINEERING

ACADEMIC YEAR 2025-2026
SEMESTER 1

EE2213: Introduction to AI

TUTORIAL 7: L10-L11

Question 1 (Lecture 10)

What type of cluster does the K-means clustering method form?

- (a) Well-separated clusters
- (b) Center-based clusters
- (c) Contiguous clusters
- (d) Density-based clusters

Answer: (b)

Question 2 (Lecture 10)

The K-means clustering method can only find local optima.

- (a) True
- (b) False

Answer: (a). Because the cost function is non-convex.

Question 3 (Lecture 10)

In order to determine an optimal K from a possible set of K values for K-means clustering, Elbow method which selects the K with the minimum within-cluster variance can be applied.

- (a) True
- (b) False

Answer: (b). Should choose K corresponding to “bend” or “knee”, instead of the minimum within-cluster variance.

Question 4 (Lecture 10)

Prove that the membership degree sums to 1 for each data point in Fuzzy C-means clustering.

Answer:

Need to prove $\sum_{k=1}^C w_{ik} = 1$

$$\begin{aligned}
\sum_{k=1}^C w_{ik} &= \sum_{k=1}^C \frac{1}{\sum_{j=1}^C \left(\frac{\|x_i - c_k\|}{\|x_i - c_j\|} \right)^{\frac{2}{r-1}}} = \sum_{k=1}^C \frac{1}{\sum_{j=1}^C \frac{(\|x_i - c_k\|)^{\frac{2}{r-1}}}{(\|x_i - c_j\|)^{\frac{2}{r-1}}}} \\
&= \sum_{k=1}^C \frac{1}{(\|x_i - c_k\|)^{\frac{2}{r-1}} \sum_{j=1}^C \frac{1}{(\|x_i - c_j\|)^{\frac{2}{r-1}}}} \\
&= \sum_{k=1}^C \frac{1}{(\|x_i - c_k\|)^{\frac{2}{r-1}}} \frac{1}{\sum_{j=1}^C \frac{1}{(\|x_i - c_j\|)^{\frac{2}{r-1}}}} \\
&= \sum_{k=1}^C \frac{1}{(\|x_i - c_k\|)^{\frac{2}{r-1}}} \frac{1}{\sum_{k=1}^C \frac{1}{(\|x_i - c_k\|)^{\frac{2}{r-1}}}} = 1
\end{aligned}$$

Question 5 (Lecture 10)

Consider the following data points: $\mathbf{x}_1 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$, $\mathbf{x}_2 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$, $\mathbf{x}_3 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$, $\mathbf{x}_4 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$. Both the K-means and Fuzzy C-means clustering are initialized with centers at $\mathbf{x}_1$ and $\mathbf{x}_2$.

- What are the two cluster centers found by K-means clustering upon convergence, or after a maximum of 100 iterations?
- What are the two cluster centers found by Fuzzy C-means clustering upon convergence, or after a maximum of 100 iterations? Assume fuzzier $r=2$.

Answer: See TUTORIAL_Q5.ipynb

- Initialization: $\mathbf{c}_1 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$, $\mathbf{c}_2 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$

1st iteration: Assignment:

$$\begin{aligned}
&\text{dist}(\mathbf{x}_1, \mathbf{c}_1) = 0, \text{dist}(\mathbf{x}_1, \mathbf{c}_2) = 1, \mathbf{x}_1 \in \text{Cluster 1} \\
&\text{dist}(\mathbf{x}_2, \mathbf{c}_1) = 1, \text{dist}(\mathbf{x}_2, \mathbf{c}_2) = 0, \mathbf{x}_2 \in \text{Cluster 2} \\
&\text{dist}(\mathbf{x}_3, \mathbf{c}_1) = 1, \text{dist}(\mathbf{x}_3, \mathbf{c}_2) = 1.41, \mathbf{x}_3 \in \text{Cluster 1} \\
&\text{dist}(\mathbf{x}_4, \mathbf{c}_1) = 1.41, \text{dist}(\mathbf{x}_4, \mathbf{c}_2) = 1, \mathbf{x}_4 \in \text{Cluster 2}
\end{aligned}$$

$$\text{Centroid Update: } \mathbf{c}_1 = \frac{1}{2}(\mathbf{x}_1 + \mathbf{x}_3) = \begin{bmatrix} 0 \\ 0.5 \end{bmatrix}, \mathbf{c}_2 = \frac{1}{2}(\mathbf{x}_2 + \mathbf{x}_4) = \begin{bmatrix} 1 \\ 0.5 \end{bmatrix}$$

2nd iteration: Assignment:

$$\begin{aligned}
&\text{dist}(\mathbf{x}_1, \boldsymbol{\mu}_1) = 0.5, \text{dist}(\mathbf{x}_1, \boldsymbol{\mu}_2) = 1.12, \mathbf{x}_1 \in \text{Cluster 1} \\
&\text{dist}(\mathbf{x}_2, \boldsymbol{\mu}_1) = 1.12, \text{dist}(\mathbf{x}_2, \boldsymbol{\mu}_2) = 0.5, \mathbf{x}_2 \in \text{Cluster 2} \\
&\text{dist}(\mathbf{x}_3, \boldsymbol{\mu}_1) = 0.5, \text{dist}(\mathbf{x}_3, \boldsymbol{\mu}_2) = 1.12, \mathbf{x}_3 \in \text{Cluster 1} \\
&\text{dist}(\mathbf{x}_4, \boldsymbol{\mu}_1) = 1.12, \text{dist}(\mathbf{x}_4, \boldsymbol{\mu}_2) = 0.5, \mathbf{x}_4 \in \text{Cluster 2} \\
&\text{Cluster assignment is unchanged. Converged.}
\end{aligned}$$

$$\text{Two cluster centers found: } \mathbf{c}_1 = \begin{bmatrix} 0 \\ 0.5 \end{bmatrix}, \mathbf{c}_2 = \begin{bmatrix} 1 \\ 0.5 \end{bmatrix}$$

- Two cluster centers found: $\mathbf{c}_1 = \begin{bmatrix} 0.0449 \\ 0.4999 \end{bmatrix}$, $\mathbf{c}_2 = \begin{bmatrix} 0.9551 \\ 0.4999 \end{bmatrix}$

$$\text{Membership matrix } \mathbf{W} = \begin{bmatrix} 0.8219 & 0.1781 \\ 0.1781 & 0.8219 \\ 0.8217 & 0.1783 \\ 0.1783 & 0.8217 \end{bmatrix}$$

Question 6 (Lecture 10)

Load the wine dataset by “from sklearn.datasets import load_wine”. Assume that the class labels are not given. Apply K-means clustering algorithm to group all the data with different K from 2 to 10. Which K is the best according to the Elbow method?

Answer: See TUT7_Q6.ipynb

Choose the K with “bend” or “knee” (Abrupt change in the slop): either 3 or 4.

Question 7 (Lecture 11)

In neural networks, what is the purpose of introducing nonlinear activation functions like sigmoid and ReLU?

- (a) Ensure the output values to be between 0 and 1.
- (b) Enable the model to learn complex patterns.
- (c) Speed up the gradient calculation in backpropagation.

Answer: (b)

Question 8 (Lecture 11)

A multi-layer perceptron (MLP) of 2 layers has been constructed as:

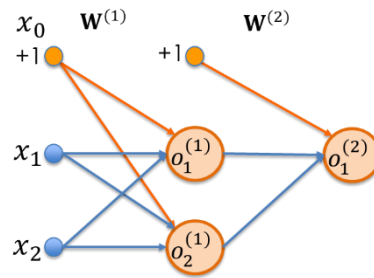
$$f_{\mathbf{W}}(\mathbf{x}) = [1, \sigma(\mathbf{x}^T \mathbf{W}^{(1)})] \mathbf{W}^{(2)}$$

where σ is the ReLU function, $\mathbf{W}^{(1)} = \begin{bmatrix} -1 & 0 \\ 0 & 1 \\ 1 & -1 \end{bmatrix}$, $\mathbf{W}^{(2)} = \begin{bmatrix} 1 \\ -1 \\ 1 \end{bmatrix}$.

Suppose that this MLP is used to train a dataset $\mathbf{X} = \begin{bmatrix} 1 & 2 & 1 \\ 1 & 5 & 1 \end{bmatrix}$ with the target $\mathbf{y} = \begin{bmatrix} 0.1 \\ 0.7 \end{bmatrix}$. The cost function is $J(\mathbf{W}) = \frac{1}{N} \sum_{i=1}^N (f_{\mathbf{W}}(\mathbf{x}_i) - y_i)^2$ for optimization. The learning rate η is 0.1. What will $\mathbf{W}^{(1)}$ and $\mathbf{W}^{(2)}$ be after the 1st iteration of backpropagation?

Answer: See TUT_Q8.ipynb

1. Architecture of MLP:



2. Learning:

1st iteration: Step 1: Forward Pass:

$$\begin{aligned}
 \mathbf{f}_{\mathbf{W}(0)}(\mathbf{X}) &= \left[\mathbf{1}, \sigma^{(1)} \left(\mathbf{X} \mathbf{W}^{(1)}(0) \right) \right] \mathbf{W}^{(2)}(0) \\
 &= \left[\mathbf{1}, \text{ReLU} \left(\begin{bmatrix} 1 & 2 & 1 \\ 1 & 5 & 1 \end{bmatrix} \begin{bmatrix} -1 & 0 \\ 0 & 1 \\ 1 & -1 \end{bmatrix} \right) \right] \begin{bmatrix} 1 \\ -1 \\ 1 \end{bmatrix} \\
 &= \begin{bmatrix} 1 & 0 & 1 \\ 1 & 0 & 4 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \\ 1 \end{bmatrix} \\
 &= \begin{bmatrix} 2 \\ 5 \end{bmatrix}
 \end{aligned}$$

Step 2: Backward Pass:

$$\mathcal{L} = (f_{\mathbf{W}}(\mathbf{x}) - y)^2$$

(1) Output layer:

formula from the lecutre notes: $\frac{\partial \mathcal{L}}{\partial \mathbf{w}_{i,j}^{(L)}} = \epsilon_j^{(L)} a_i^{(L)},$

where $\epsilon_j^{(L)} = \sum_{m=1}^{D_L} \mathcal{L}'(o_m^{(L)}) \sigma^{(L)'}(z_j^{(L)})_m$

$$\epsilon_1^{(2)} = \mathcal{L}'(o_1^{(2)}) = \mathcal{L}'(f_{\mathbf{W}}(\mathbf{x})) = 2(f_{\mathbf{W}}(\mathbf{x}) - y)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(2)}} = 2(f_{\mathbf{W}}(\mathbf{x}) - y) \mathbf{a}^{(2)}$$

$$\frac{dJ}{d\mathbf{W}^{(2)}} = \frac{1}{N} \sum_{i=1}^N \frac{\partial \mathcal{L}_i}{\partial \mathbf{W}^{(2)}} = \frac{1}{N} \sum_{i=1}^2 2(f_{\mathbf{W}}(\mathbf{x}_i) - y_i) \mathbf{a}^{(2)} = \frac{1}{N} (\mathbf{A}^{(2)})^T 2(\mathbf{f}_{\mathbf{W}}(\mathbf{X}) - \mathbf{y})$$

$$\frac{dJ(\mathbf{W}^{(2)}(0))}{d\mathbf{W}^{(2)}} = \begin{bmatrix} 1 & 0 & 1 \\ 1 & 0 & 4 \end{bmatrix}^T \left(\begin{bmatrix} 2 \\ 5 \end{bmatrix} - \begin{bmatrix} 0.1 \\ 0.7 \end{bmatrix} \right) = \begin{bmatrix} 6 \\ 0 \\ 19.1 \end{bmatrix}$$

$$\mathbf{W}^{(2)}(1) = \mathbf{W}^{(2)}(0) - \eta \frac{dJ(\mathbf{W}^{(2)}(0))}{d\mathbf{W}^{(2)}} = \begin{bmatrix} 1 \\ -1 \\ 1 \end{bmatrix} - 0.1 \begin{bmatrix} 6 \\ 0 \\ 19.1 \end{bmatrix} = \begin{bmatrix} 0.4 \\ -1 \\ -0.91 \end{bmatrix}$$

(2) Hidden layer:

Since it is ReLU activation,

$$\frac{dJ(\mathbf{W}^{(1)}(0))}{d\mathbf{W}^{(1)}} = \frac{1}{N} (\mathbf{A}^{(1)})^T \mathbf{E}^{(1)},$$

$$\begin{aligned} \text{where } \mathbf{E}^{(1)} &= \delta_{\mathbf{Z}^{(1)}} * \left(\mathbf{E}^{(2)} (\mathbf{W}_{1:,2}^{(2)}(0))^T \right) \\ &= \begin{bmatrix} 0 & 1 \\ 0 & 1 \end{bmatrix} * \left(2 \left(\begin{bmatrix} 2 \\ 5 \end{bmatrix} - \begin{bmatrix} 0.1 \\ 0.7 \end{bmatrix} \right) [-1, 1] \right) = \begin{bmatrix} 0 & 3.8 \\ 0 & 8.6 \end{bmatrix}, \end{aligned}$$

$$\mathbf{A}^{(1)} = \mathbf{X} = \begin{bmatrix} 1 & 2 & 1 \\ 1 & 5 & 1 \end{bmatrix}$$

$$\text{therefore, } \frac{dJ(\mathbf{W}^{(1)}(0))}{d\mathbf{W}^{(1)}} = \frac{1}{2} \begin{bmatrix} 1 & 1 \\ 2 & 5 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 0 & 3.8 \\ 0 & 8.6 \end{bmatrix} = \begin{bmatrix} 0 & 6.2 \\ 0 & 25.3 \\ 0 & 6.2 \end{bmatrix}$$

$$\begin{aligned} \text{then, } \mathbf{W}^{(1)}(1) &= \mathbf{W}^{(1)}(0) - \eta \frac{dJ(\mathbf{W}^{(1)}(0))}{d\mathbf{W}^{(1)}} \\ &= \begin{bmatrix} -1 & 0 \\ 0 & 1 \\ 1 & -1 \end{bmatrix} - 0.1 \begin{bmatrix} 0 & 6.2 \\ 0 & 25.3 \\ 0 & 6.2 \end{bmatrix} = \begin{bmatrix} -1 & -0.62 \\ 0 & -1.53 \\ 1 & -1.62 \end{bmatrix} \end{aligned}$$

Question 9 (Lecture 11)

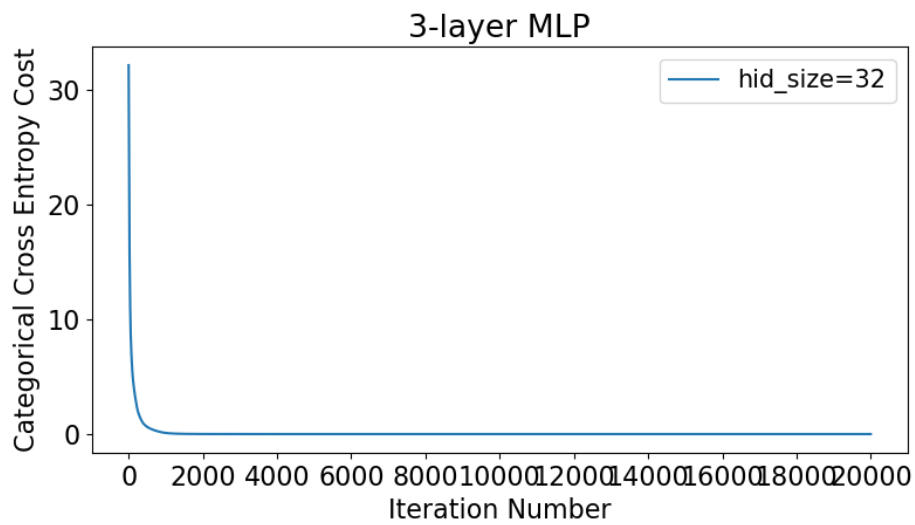
The Digits dataset is a classic, lightweight dataset ideal for testing simple machine learning models like MLPs, especially on CPU. It has 10 classes (digits 0 to 9). Each sample is a grayscale image with a dimension of 8×8 . Get the dataset by “from sklearn.datasets import load_digits”. Use Python to do the following tasks.

- (i) Split the database into three sets: 70% of samples for training, 15% of samples for validation, and 15% of samples for testing. Normalize the features using z-score standardization. Hint: you may use “from sklearn.preprocessing import StandardScaler”.
- (ii) Construct the target output using one-hot encoding.
- (iii) Use a 3-layer MLP for the digit classification. Suppose that ReLU is the activation function for the hidden layers and Softmax is the activation function for the output layer. Assume that the number of the neurons for each hidden layer is 32. The learning rate is 0.01. The number of iterations for training is 20000. The objective function is $J(\mathbf{W}) = \frac{1}{N} \sum_{i=1}^N L_{CCE}(\mathbf{f}_{\mathbf{W}}(\mathbf{x}_i), \mathbf{y}_i)$, where $L_{CCE}(\mathbf{f}_{\mathbf{W}}(\mathbf{x}_i), \mathbf{y}_i)$ is the categorical cross-entropy loss. Plot the cost over number of iterations. Compute the training accuracy and validation accuracy.
- (iv) Change the hidden layer size to 36 and do the same thing as in (iii). Which one is better? Use the model with the better hidden layer size to compute the accuracy of the test set.

Answer: See TUT7_Q9

Note: Different initialization of the weight parameters and data splitting may have different results.

(iii) Training accuracy = 1, Validation accuracy = 0.8741



(iv) Training accuracy = 1, Validation accuracy = 0.8259,

Hidden layer size 32 is better according to the validation accuracy.

Test accuracy for model with hidden layer size of 32 = 0.8444.

